



depends-on **Meta-Argument**

Ensuring Resources Are Created in the Right Order

● Explicitly Define Dependencies

Terraform may not automatically detect or know about dependencies needed for your deployment

● Ensure Correct Provisioning Order

Useful for resources that rely on others to exist first

● Prevents Race Conditions

Complex deployments may require one resource to be created before another

```
1 resource "aws_db_instance" "db" {
2     identifier      = "mydb"
3     engine          = "mysql"
4     instance_class  = "db.t3.micro"
5     allocated_storage = 20
6     username        = "admin"
7     password        = "SuperSecurePassword123!"
8     skip_final_snapshot = true
9 }
10
11 resource "aws_instance" "app_server" {
12     ami            = "ami-0abcdef1234567890"
13     instance_type  = "t2.micro"
14
15     depends_on = [aws_db_instance.db]
16 }
```

Dependencies

Implicit vs. Explicit Dependency



- ✦ Implicit dependency is when one resource references another resource's attribute. When you use an expression like `aws_instance.web.id`, Terraform knows it must create the referenced resource first - no manual configuration needed.
- ✦ Sometimes resources need to be created in a specific order, but there's no attribute reference to create an implicit dependency. In these cases, you use the `depends_on` argument to manually create an explicit dependency to tell Terraform "create this other thing first."



Implicit Dependency

```
1 resource "aws_security_group" "web" {  
2   name = "web-sg"  
3 }  
4  
5 resource "aws_instance" "app" {  
6   ami          = "ami-12345"  
7   instance_type = "t2.micro"  
8   security_groups = [aws_security_group.web.id]  
9 }
```

resource reference

Explicit Dependency

```
1 resource "aws_iam_role" "lambda" {  
2   name = "lambda-role"  
3   # role configuration...  
4 }  
5  
6 resource "aws_lambda_function" "app" {  
7   function_name = "my-function"  
8   role          = aws_iam_role.lambda.arn  
9  
10  depends_on = [aws_iam_role_policy_attachment.lambda_logs]  
11 }
```

explicit depends_on



lifecycle **Meta-Argument**

Controlling How Terraform Manages Resources

- **Customizes how Terraform Manages Resources**
Change the default behavior during your Terraform workflow.
- **You can protect resources or avoid changes**
Customizes how Terraform creates, updates, and destroys specific resources.
- **Uses the `lifecycle` block inside of your resource**
Options include `prevent_destroy`, `create_before_destroy`, and `ignore_changes`

```
1  resource "azurerm_resource_group" "example" {
2      name      = "my-resource-group"
3      location = "East US"
4
5      lifecycle {
6          prevent_destroy = true
7      }
8  }
9
10 resource "azurerm_virtual_machine" "example" {
11     name              = "my-vm"
12     location           = "East US"
13     resource_group_name = "my-resource-group"
14     vm_size            = "Standard_D2s_v3"
15
16     lifecycle {
17         ignore_changes = [vm_size]
18     }
19 }
```

Lifecycle Options



main.tf

```
resource "aws_instance" "web" {  
  ami           = "ami-12345"  
  instance_type = "t2.micro"  
  
  lifecycle {  
    create_before_destroy = true  
  }  
}
```

create_before_destroy

Tells Terraform to create the replacement resource before destroying the old one.

Essential for zero-downtime updates when resources must be replaced.

Lifecycle Options



prevent_destroy

Blocks Terraform from destroying the resource.

Provides protection against accidental deletion of critical infrastructure like databases.

```
main.tf
resource "aws_db_instance" "prod" {
  identifier = "production-db"
  # ...

  lifecycle {
    prevent_destroy = true
  }
}
```

Lifecycle Options



```
main.tf
resource "aws_instance" "app" {
  ami = "ami-12345"

  tags = {
    Name = "app-server"
  }

  lifecycle {
    ignore_changes = [tags]
  }
}
```

ignore_changes

Tells Terraform to ignore changes to specific attributes.

Perfect when external systems modify resources outside of Terraform.

Lifecycle Options



replace_triggered_by

Forces resource replacement when specified resources or attributes change.

Useful for dependencies that don't trigger automatic replacement.

```
main.tf

resource "aws_instance" "web" {
  ami = "ami-12345"

  lifecycle {
    replace_triggered_by = [
      aws_security_group.web
    ]
  }
}
```


Lifecycle Options



main.tf

```
resource "azurerm_virtual_machine" "web" {  
  name      = "web-vm"  
  location  = var.location  
  vm_size   = var.vm_size  
  
  lifecycle {  
    precondition {  
      condition      = var.vm_size != "Basic_A0"  
      error_message = "Basic_A0 is deprecated"  
    }  
  }  
}
```

precondition
postcondition

**Validate resource state
before/after operations.**

**Advanced validation for complex
deployments.**



Validating Your Configuration

Terraform has four validation mechanisms, each serving different purposes

- Variable Validation
- Preconditions
- Postconditions
- Checks

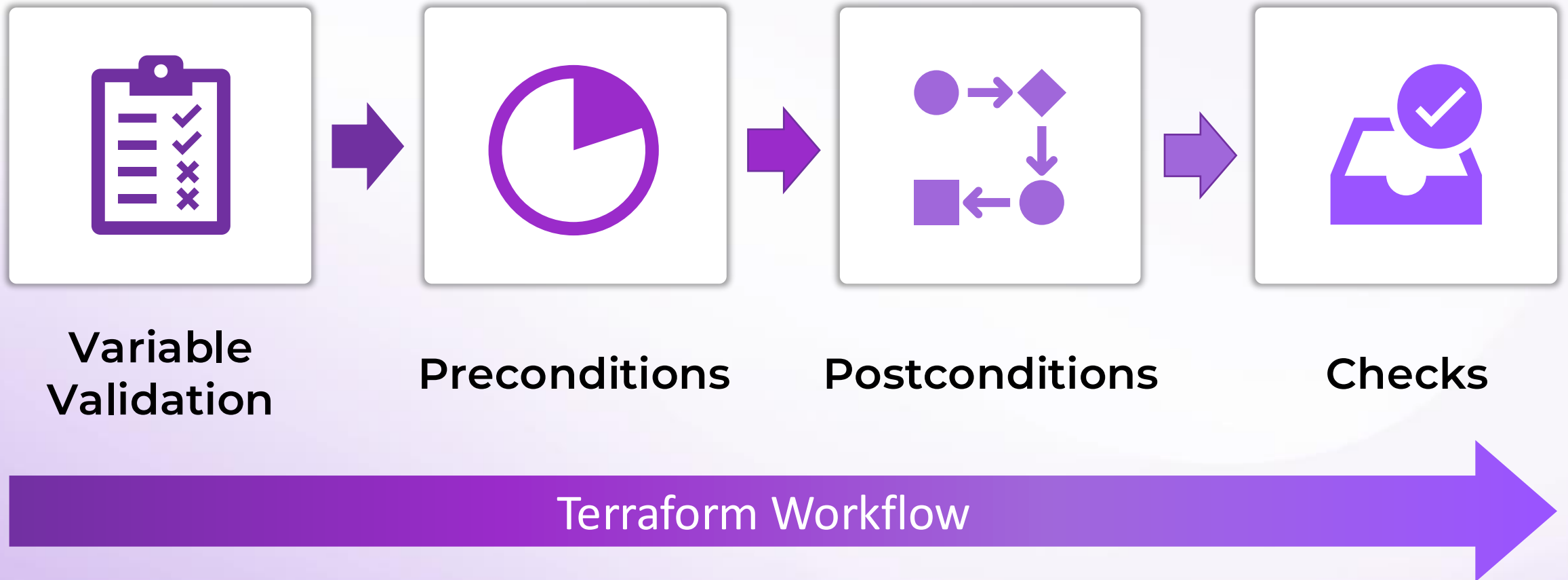


Think of these as defensive layers protecting your infrastructure at different stages



Validating Your Configuration

Order of Operations





Variable Validation

Verify your configuration parameters when Terraform creates a plan

- ❏ Verify input variables meet specific format requirements
- ❏ Verify input values fall within acceptable ranges
- ❏ Prevent Terraform operations if a variable is misconfigured
- ❏ Failure results in displaying the configured `error_message`, and stops the operation from proceeding

```
1 variable "variable_name" {  
2   value = <expression-to-evaluate>  
3  
4   validation {  
5     condition      = <expression-to-verify>  
6     error_message = "<error message string>"  
7   }  
8 }
```

Configured inside the `variable` block
you want validated



Variable Validation

Verify your configuration parameters when Terraform creates a plan

```
1 variable "location" {
2     type      = string
3     description = "Azure region where resources will be deployed"
4
5     validation {
6         condition = contains([
7             "eastus",
8             "westus2",
9             "centralus"
10        ], var.location)
11         error_message = "Location must be one of the approved US regions: eastus, westus2, or centralus."
12     }
13 }
```



Variable Validation

Verify your configuration parameters when Terraform creates a plan

```
1 variable "cpu_count" {
2     type      = number
3     description = "Number of virtual CPUs for the VM"
4
5     validation {
6         condition     = var.cpu_count >= 1 && var.cpu_count <= 16 && var.cpu_count % 2 == 0
7         error_message = "CPU count must be an even number between 2 and 16. Single CPU VMs are not supported in production."
8     }
9 }
```


Variable Validation



Verify your configuration parameters when Terraform creates a plan

```
1 variable "instance_type" {
2   type      = string
3   description = "EC2 instance type for the application server"
4
5   validation {
6     condition = contains([
7       "t3.micro", "t3.small", "t3.medium",
8       "t3.large", "t3.xlarge", "t3.2xlarge",
9       "m5.large", "m5.xlarge", "m5.2xlarge",
10      "c5.large", "c5.xlarge", "c5.2xlarge"
11    ], var.instance_type)
12    error_message = "Instance type must be an approved type from the t3, m5, or c5 families. Graviton instances require architecture approval."
13  }
14
15  validation {
16    condition = (
17      var.instance_type == "t3.micro" ||
18      var.instance_type == "t3.small" ||
19      !startswith(var.instance_type, "t3")
20    )
21    error_message = "T3.micro and t3.small are only allowed for development environments. Use t3.medium or larger for staging and production."
22  }
23
24  validation {
25    condition = !can(regex("\\.(24|32)xlarge$", var.instance_type))
26    error_message = "Extra-large instances (24xlarge and above) require infrastructure team approval. Please submit a request ticket."
27  }
28 }
```



Preconditions/Postconditions

Verify your resources before and after Terraform creates them

Preconditions:

- Verify BEFORE Terraform creates something (e.g., checking that an AMI has the right architecture before launching an instance)

Postconditions:

- Verify AFTER Terraform creates something (e.g., ensuring an instance actually got a private DNS record after launch)

Configured inside the `lifecycle` block for the resource you want to verify

```
1 resource {
2   lifecycle {
3     precondition {
4       condition = <expression>
5       error_message = "<message>"
6     }
7   }
8 }
```

```
1 resource {
2   lifecycle {
3     postcondition {
4       condition = <expression>
5       error_message = "<message>"
6     }
7   }
8 }
```



Preconditions/Postconditions

Verify your resources before and after Terraform creates them



```
1 resource "azurerm_app_service" "webapp" {
2     name                = var.app_service_name
3     location             = var.location
4     resource_group_name = var.resource_group_name
5     app_service_plan_id = azurerm_app_service_plan.plan.id
6
7     lifecycle {
8         precondition {
9             condition      = contains(data.azurerm_subnet.app_subnet.service_endpoints, "Microsoft.Web")
10            error_message = "The subnet must have the Microsoft.Web service endpoint enabled before deploying App Services."
11        }
12    }
13 }
```



Preconditions/Postconditions

Verify your resources before and after Terraform creates them

```
1 resource "google_compute_instance" "web_server" {
2     name          = var.instance_name
3     machine_type  = var.machine_type
4     zone          = var.zone
5
6     boot_disk {
7         initialize_params {
8             image = var.boot_image
9         }
10    }
11
12    lifecycle {
13        postcondition {
14            condition      = self.network_interface[0].access_config[0].nat_ip != ""
15            error_message = "The instance must have a public IP address assigned. Check your network configuration."
16        }
17    }
18 }
```

Checks (via check Blocks)

Validate your infrastructure outside of the typical resource lifecycle



- ✓ Validate resources, data sources, variables, or outputs in your configuration.
- ✓ Validate the behavior of your infrastructure as a whole.
- ✓ Verify infrastructure configuration without blocking operations.
- ✓ The `check` block executes as the last step of `plan` or `apply` operation, after Terraform has planned or provisioned your infrastructure.
- ✓ When a `check` block's assertion fails, Terraform reports a warning and continues executing the current operation.



Check blocks are standalone, top-level blocks in your `.tf` files - they're NOT nested inside resources, data sources, or any other block type.

Checks (via check Blocks)



Validate your infrastructure outside of the typical resource lifecycle

```
1  check "web_health" {
2    data "http" "web_health_endpoint" {
3      url = "https://my-awesome.website.com/health"
4    }
5
6    assert {
7      condition      = data.http.web_health_endpoint.status_code == 200
8      error_message = "Web health endpoint returned ${data.http.web_health_endpoint.status_code}. Expected 200."
9    }
10 }
```

Choosing the Right Validation



What Do You Need To Do?

Validation Type

- ✧ Validate user input before planning? → **Variable validation**
- ✧ Verify assumptions before creating a resource? → **Precondition**
- ✧ Verify results after creating a resource? → **Postcondition**
- ✧ Monitor infrastructure health without blocking operations? → **Check**